Investigation and Analysis of Computing Data for Data Management

Nishnat Acharya

S1-s1-E39C

Bsc Hons Ethical Hacking & Cyber Security, Softwarica College of IT & E-commerce

ST4015CMD Foundations of Computer Science

Submitted to Rupak Rajbansi

5th Mar 2026

Table of Contents

Table of Contents **2**
Task 1: Enhancing Secure Data Exchange with Encoding Formats and Protocol Integration
1.1 What is Encoding and Why Does it Matter? 4
1.2 Strengths and Weaknesses of Key Encoding Formats 4
1.3 Data Flow: Base64 in TLS-Based Email Transmission 5
1.4 Encoding in HTTP Transmission and Injection Attack Prevention 6
1.5 Interoperability with REST APIs and OAuth 6
1.6 Risks of Encoding-Based Obfuscation by Attackers 6
1.7 Proposed Enhanced Encoding Strategies 6
Task 2: Classroom Seating Arrangement Problem, P vs NP and Algorithmic Approaches **8**
2.1 What Are P and NP Problems? 8
2.2 Applying P vs NP to the Seating Problem 8
2.3 Brute Force Approach 9
2.4 Heuristic (Smart) Approach 9
Task 3: College Club Membership Management and Database Normalisation **10**
3.1 The Original Table and Its Problems 10
3.2 Normalisation: Making the Data Better 11
3.3 Entity Relationship Diagram 12
3.4 Basic SQL Operations 12
Task 4: Insert and Select Queries 13
Task 5: SQL JOIN Query 13
3.5 Reflection: Benefits of Normalisation and the Need for JOINs 13
References **15**
Appendix: Project Repository **16**

List of Figures

Figure 1: How Base64 Encoding Works (source: original diagram)..... 5
Figure 2: Comparison of Common Encoding Formats..... 5
Figure 3: Base64 and TLS Email Transmission Flow (source: original diagram).....6
Figure 4: Comparison of P and NP Problem Classes.....8
Figure 6: Growth of Brute Force Complexity with Number of Students.....9
Figure 8: Database Normalisation Stages from 0NF through to 3NF (source: original diagram)..... 10
Figure 9: Original Single-Table Design for Club Membership..... 11
Figure 11: Normalised Student Table (2NF)..... 11
Figure 12: Normalised Club Table (2NF)..... 12
Figure 13: Normalised Membership Table (2NF).....12
Figure 14: Entity Relationship Diagram for Club Membership System..... 12
Figure 15: SQL JOIN Operations and Reassembly of Normalised Data (source: original diagram)..... 13

Task 1: Enhancing Secure Data Exchange with Encoding Formats and Protocol Integration

Data encoding is one of the foundational ideas that keeps the modern internet running smoothly. Whenever you log into a website, send an email, or upload a file, the data you send passes through a process of encoding before it is transmitted. According to Built In, Base64 is a binary-to-text encoding scheme used to convert binary data into ASCII text format, so it can be safely transmitted over communication channels that expect text, such as email or web forms. This section explores the most important encoding formats, Base64, Hex, and URL encoding, and examines how they work together with protocols such as HTTPS, TLS, and SMTP to enable secure and efficient communication.

1.1 What is Encoding and Why Does it Matter?

Encoding is the process of converting data from one format into another so that it can be safely transmitted or stored. It is important to understand that encoding is not the same as encryption. Encryption hides the meaning of data, whereas encoding simply changes the representation. Anyone who knows the encoding scheme can reverse the process and recover the original data, as noted by the MIME specification going back to RFC 1341 published in 1992, which officially introduced the Base64 standard.

Encoding matters because different systems and protocols were built with different assumptions about what kind of data they can handle. Email systems were designed to carry plain text. According to research published on Mutant Mail, the SMTP transmission protocol for email could originally only transmit 7-bit ASCII codes, which meant binary data such as images or attachments would be corrupted in transit without a suitable encoding layer. This is where Base64 became essential.

1.2 Strengths and Weaknesses of Key Encoding Formats

Base64

Base64 converts binary data into a string of 64 printable ASCII characters by splitting the raw input into 24-bit blocks, then dividing each block into four 6-bit segments that are mapped onto the alphabet of 64 characters. This alphabet consists of the 26 uppercase letters A through Z, the 26 lowercase letters a through z, the digits 0 through 9, and the two special characters plus and forward slash. According to Wikipedia, Base64 encoding enables transmitting binary data on a communication channel that only supports text, and this is precisely why it became integral to email, JWTs, and PEM certificates.

The main drawback of Base64 is the size overhead. Every three bytes of input produce four bytes of output, representing a 33 percent expansion. A 100 KB image becomes roughly 133 KB after encoding. It also provides no confidentiality: any attacker who intercepts the data and knows it is Base64-encoded can decode it immediately.

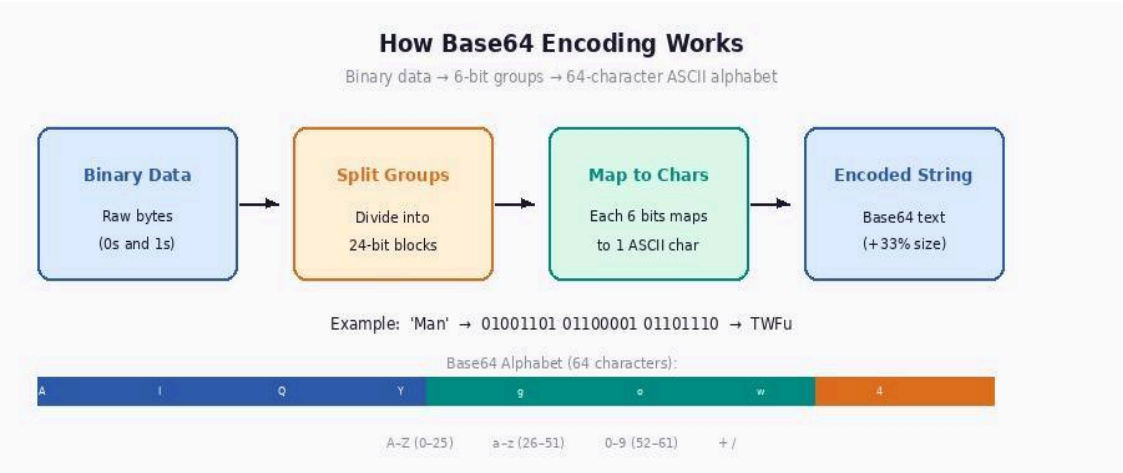

Figure 1: How Base64 Encoding Works (source: original diagram)

Hexadecimal (Hex) Encoding

Hex encoding converts each byte of data into two hexadecimal characters drawn from the set 0 through 9 and A through F. The letter A, for example, has ASCII value 65 and becomes the hex string 41. This format is extremely readable for debugging and for representing cryptographic values such as hash outputs, certificate fingerprints, and key values. Its principal limitation is that it doubles the size of the data.

URL Encoding (Percent Encoding)

URLs can only carry a limited set of characters. Special characters such as spaces, ampersands, and equals signs would break the structure of a URL if included directly. URL encoding, also called percent encoding, replaces these characters with a percent sign followed by their two-digit hexadecimal code. A space becomes %20 and a forward slash becomes %2F. According to the W3C MIME specification, this ensures that query parameters and paths are transmitted correctly across all web systems.

Format	Typical Use	Size Overhead	Security Level
Base64	Email attachments, HTTP Auth, JWTs	+33%	None (reversible)
Hexadecimal	Crypto hashes, debugging, checksums	+100%	None (reversible)
URL Encoding	Query strings, web form data	Variable	None (reversible)

Figure 2: Comparison of Common Encoding Formats

1.3 Data Flow: Base64 in TLS-Based Email Transmission

To understand how encoding fits into real-world secure communication, consider the journey of an email with an attachment. At the first step, the user composes an email and attaches a file. The email client reads the file as raw binary bytes. At the second step, the MIME standard requires the attachment to be encoded using Base64 before it can be placed inside the email body, because SMTP was designed for plain text. The Base64-encoded string is placed in the email MIME section with a header such as Content-Transfer-Encoding: base64. At the third step, the email client and server perform a TLS handshake, verifying each other's identity using certificates and agreeing on a symmetric encryption key. At the fourth step, the entire email including the Base64-encoded attachment is encrypted using AES or another symmetric cipher, and the encrypted data then travels over the network. The receiving server decrypts it, then decodes the Base64 to recover the original file.

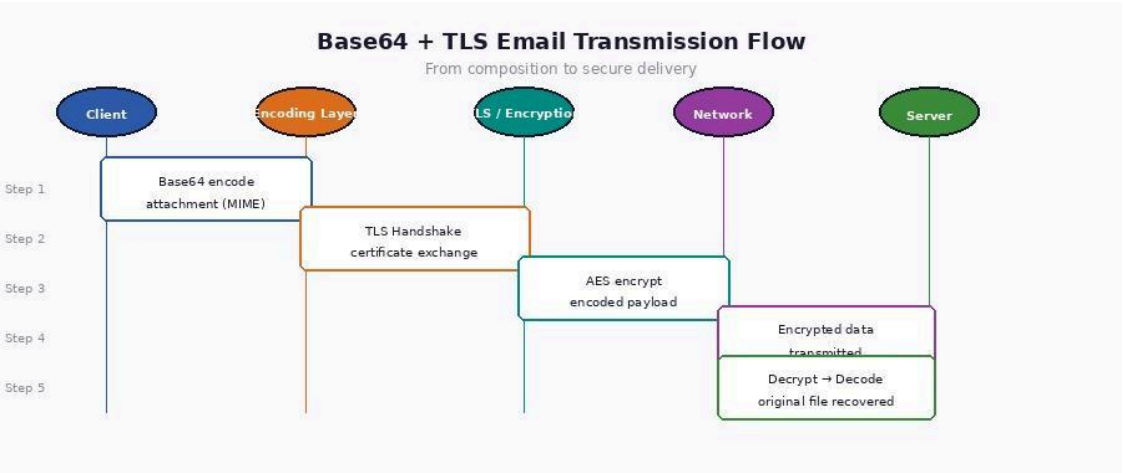

Figure 3: Base64 and TLS Email Transmission Flow (source: original diagram)

1.4 Encoding in HTTP Transmission and Injection Attack Prevention

When a web browser submits a form, the data in the form fields is URL-encoded before being sent in an HTTP POST request. A search for 'laptop and accessories' becomes 'laptop+%26+accessories' in the request body, ensuring that special characters do not corrupt the request structure. Beyond correct transmission, URL encoding also plays a key role in preventing injection attacks. The OWASP Top 10 (2021) identifies injection as one of the most critical web application security risks, noting that when user input is properly encoded before being included in a query string or database command, characters that would otherwise be interpreted as control sequences are rendered harmless.

1.5 Interoperability with REST APIs and OAuth

Modern web applications rely heavily on REST APIs to communicate between services, and encoding formats are used throughout these interactions. JSON Web Tokens, used in OAuth 2.0 authentication, consist of three Base64-URL encoded sections: header, payload, and signature. As documented in RFC 7515, Base64-URL encoding is a variant of standard Base64 that replaces the plus and forward slash characters with hyphens and underscores, making it safe for use in URLs without further percent-encoding. When a user logs in with OAuth, the authorisation server generates a JWT, encodes it in Base64-URL, and returns it to the client. The server subsequently decodes it and verifies the signature to authenticate each request. This demonstrates how encoding directly enables stateless, scalable authentication across distributed systems.

1.6 Risks of Encoding-Based Obfuscation by Attackers

While encoding is legitimate and necessary, attackers also exploit it to hide malicious payloads and evade detection systems. A common technique is to encode malware or attack scripts in Base64 and embed them in web pages or email bodies. When the victim's browser or email client decodes the Base64, it executes the malicious code. Security tools that only scan for known attack signatures in plain text will miss the encoded version. Attackers also use multiple layers of encoding, such as Base64 inside URL encoding inside HTML entities, to make detection even harder. This layering approach is designed to exhaust automated scanners and bypass signature-based intrusion detection systems.

1.7 Proposed Enhanced Encoding Strategies

> Research Note: The following recommendations draw on established standards including RFC 8471 (Token Binding), RFC 8446 (TLS 1.3), and OWASP guidelines for injection prevention. These are not theoretical suggestions but active industry practices.

To improve the security of encoding-based data exchange, several strategies are recommended. Content-aware encoding validation should be deployed at the firewall level: web application

firewalls should decode and inspect all incoming data across multiple encoding layers before applying security rules, rather than scanning only the surface representation. Strict MIME type enforcement means servers should reject email attachments or API payloads that do not match their declared content type, reducing the risk of disguised malicious files.

Token binding for JWTs provides another layer of defence. According to RFC 8471, OAuth tokens should be cryptographically bound to the TLS session, preventing token theft and replay attacks. Finally, where bandwidth is a concern, Base64-encoded data should be compressed using gzip before encoding, which partially offsets the 33 percent overhead that Base64 introduces.

Task 2: Classroom Seating Arrangement Problem, P vs NP and Algorithmic Approaches

The classroom seating problem described in this task is a compelling example of a constraint satisfaction problem. It looks simple on the surface, just arrange a handful of students in seats, but as the number of students grows it becomes surprisingly hard to solve. This section explores the theoretical computer science concepts of P and NP, applies them to the seating scenario, and then examines both a brute-force approach and a smarter heuristic method.

2.1 What Are P and NP Problems?

In computer science, problems are classified based on how difficult they are for a computer to solve. According to the Wikipedia article on computational complexity, the class P consists of all decision problems solvable on a deterministic sequential machine in a duration polynomial in the size of the input. In practical terms, these are problems a computer can solve efficiently. Sorting a list of names is a classic P problem: with n names it takes roughly n log n steps, which grows manageably even for large n.

The class NP consists of all decision problems whose positive solutions are verifiable in polynomial time given the right information. Importantly, finding the solution may be very hard, but verifying it is easy. GeeksforGeeks illustrates this with a workplace analogy: assigning 200 employees to rooms while avoiding pairs known to conflict is an NP problem because a proposed assignment is quick to check but extremely difficult to generate from scratch. The famous unsolved question in mathematics and computer science is whether P equals NP. As of 2019, 88 percent of researchers surveyed believed P is not equal to NP, and the question remains one of the seven Millennium Prize Problems carrying a one million dollar award from the Clay Mathematics Institute.

Feature	P Problems	NP Problems
Finding solution	Fast (polynomial time)	Potentially very slow
Verifying solution	Fast (polynomial time)	Fast (polynomial time)
Tractable for large n?	Yes	Generally no
Real-world examples	Sorting, shortest path, matrix	SAT, graph colouring, seating

Figure 4: Comparison of P and NP Problem Classes

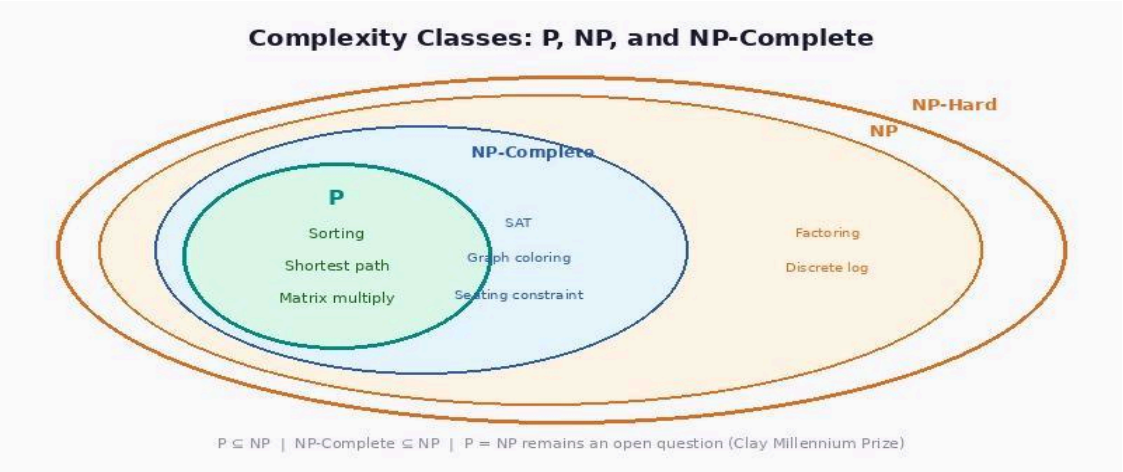

Figure 5: Complexity Class Hierarchy showing P inside NP-Complete inside NP (source: original diagram)

2.2 Applying P vs NP to the Seating Problem

Checking whether a given seating arrangement is valid is straightforward. The teacher goes through each pair of adjacent seats and checks two conditions: are these two students friends,

and are they from the same city? With n students in a row, there are n minus 1 adjacent pairs to check, which can be done in linear time O(n). Checking is therefore a P-type operation. Finding a valid arrangement from scratch is a completely different matter. With just 5 students there are 120 possible arrangements. With 10 students there are 3,628,800. With 20 students the number exceeds 2.4 times 10 to the power of 18, a figure larger than the estimated number of seconds since the Big Bang. The seating problem is closely related to well-known NP problems such as Graph Colouring and the Hamiltonian Path problem, depending on how the constraints are modelled.

2.3 Brute Force Approach

The most straightforward way to solve the seating problem is by brute force: generate every possible arrangement of students and check each one against the rules. According to Sipser's Introduction to the Theory of Computation, this exhaustive search has factorial time complexity O(n!), which grows far faster than even exponential time. The table below illustrates how rapidly the search space becomes impractical.

Students (n)	Arrangements (n!)	Time to Check All at 1 million per second
5	120	Less than 1 second
10	3,628,800	Approximately 3.6 seconds
15	1,307,674,368,000	Approximately 15 days
20	2.4 x 10^18	Approximately 77,000 years
25	1.5 x 10^25	Longer than the age of the universe

Figure 6: Growth of Brute Force Complexity with Number of Students

2.4 Heuristic (Smart) Approach

Since the brute force method becomes impractical for large classes, a smarter strategy is needed. A heuristic is a practical problem-solving technique that does not guarantee the optimal solution but finds a good-enough solution much more quickly. The first heuristic is the Most Constrained First approach, placing the student with the most constraints first. A student with many friends in the class is the hardest to seat because there are many people they cannot sit next to. By placing the most constrained student first, the algorithm avoids getting stuck later. This is similar to the fail-first principle used in constraint programming. The second heuristic involves separating same-city students early by spreading them out at the start of the process.

Step	Action
Step 1	List all students with their friend-count and city
Step 2	Pick student with most friends and place them in seat 1
Step 3	Place next student who is not a friend and not from the same city
Step 4	Continue greedy placement from left to right
Step 5	If stuck, backtrack one seat and try the next candidate
Step 6	Output the arrangement if all seats are filled without conflict

Figure 7: Greedy Heuristic Algorithm for Classroom Seating

The greedy heuristic runs in roughly O(n squared) time in the worst case, which is vastly better than O(n!). For 100 students, O(n squared) is 10,000 operations, whereas factorial time produces a number so large it has no practical meaning. The limitation of heuristics is that they are not guaranteed to find a valid arrangement even when one exists. However, for most real classroom scenarios, a greedy heuristic will find an acceptable arrangement quickly and efficiently. As Garey and Johnson note in Computers and Intractability, this kind of approximation strategy is the standard practical response to NP-hard problems.

Task 3: College Club Membership Management and Database Normalisation

Good data management is at the heart of reliable information systems. According to DigitalOcean, database normalisation is a step-by-step approach to structuring data in a way that reduces redundancy and preserves data integrity. The process is organised into a series of normal forms, 1NF, 2NF, 3NF, and BCNF, each designed to resolve specific types of data anomalies and structural problems. This section analyses the problems in the given club membership table and shows how normalisation resolves these issues.

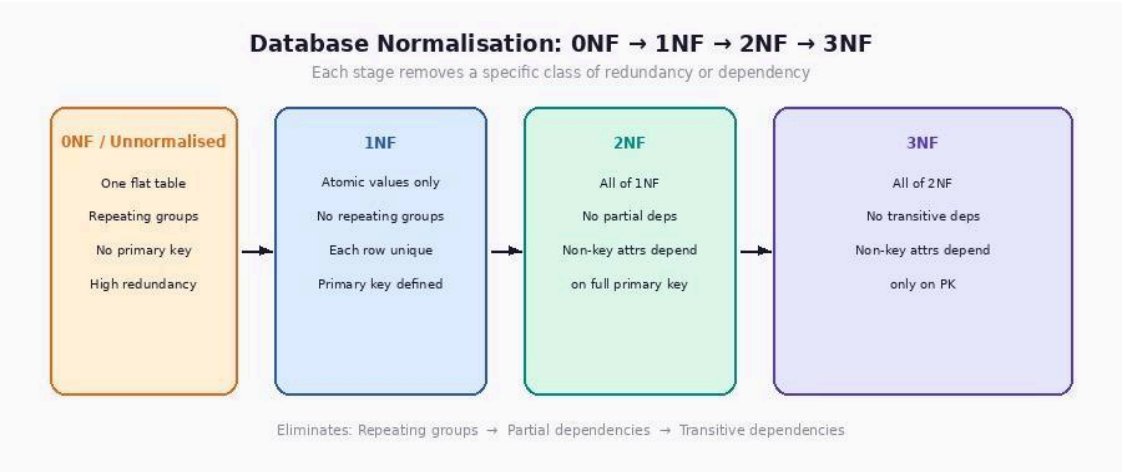

Figure 8: Database Normalisation Stages from 0NF through to 3NF (source: original diagram)

3.1 The Original Table and Its Problems

The college currently stores all club-related information in a single flat table. This design illustrates the kind of unnormalised structure that Edgar F. Codd set out to eliminate when he proposed the relational model in 1970. As Wikipedia notes, Codd introduced the concept of normalisation specifically to free collections of relations from undesirable insertion, update, and deletion dependencies.

StudentID	StudentName	Email	ClubName	ClubRoom	ClubMentor	JoinDate
1	Asha	asha@email.com	Music Club	R101	Mr. Raman	1/10/2024
2	Bikash	bikash@email.com	Sports Club	R202	Ms. Sita	1/12/2024
1	Asha	asha@email.com	Sports Club	R202	Ms. Sita	1/15/2024
3	Nisha	nisha@email.com	Music Club	R101	Mr. Raman	1/20/2024
4	Rohan	rohan@email.com	Drama Club	R303	Mr. Kiran	1/18/2024
5	Suman	suman@email.com	Music Club	R101	Mr. Raman	1/22/2024
2	Bikash	bikash@email.com	Drama Club	R303	Mr. Kiran	1/25/2024
6	Pooja	pooja@email.com	Sports Club	R202	Ms. Sita	1/27/2024

3	Nisha	nisha@email.com	Coding Club	Lab1	Mr. Anil	1/28/2024
7	Aman	aman@email.com	Coding Club	Lab1	Mr. Anil	1/30/2024

Figure 9: Original Single-Table Design for Club Membership

Three categories of problems arise from this design. The first is data redundancy. Asha appears in two rows because she has joined two clubs, meaning her name, email, and StudentID are repeated unnecessarily. Music Club information appears three times. The second problem is the update anomaly. If Mr. Raman is replaced as the Music Club mentor, the change must be made in all three rows. Missing even one creates inconsistency across the database. The third category covers insert and deletion anomalies. A new Photography Club cannot be added without attaching it to a student, and if Rohan leaves the Drama Club his row is deleted along with the only record of the Drama Club room and mentor.

Anomaly Type	Description	Example from the Table
Redundant Data	Same data stored multiple times	Music Club info repeated 3 times
Update Anomaly	Updating one row but not others	Changing mentor requires 3 updates
Insert Anomaly	Cannot add data without unrelated data*Figure 10: Summary of Data Problems in the Original Table*	Cannot add Photography Club without a student
Deletion Anomaly	Deleting a row removes unrelated data	Deleting Rohan removes Drama Club info

3.2 Normalisation: Making the Data Better

Normalisation is the process of organising a database to reduce redundancy and improve data integrity. According to GeeksforGeeks, normal forms are a set of progressive rules for relational schemas that reduce redundancy and prevent data anomalies. Each normal form is stricter than the previous one: meeting a higher normal form implies the lower ones are satisfied.

A table is in First Normal Form (1NF) if every cell contains a single atomic value, every column has a unique name, and there are no repeating groups. The original table already satisfies atomicity, but there is no clear primary key. The combination of StudentID and ClubName is unique and therefore serves as a composite primary key, placing the table in 1NF. A table is in Second Normal Form (2NF) if it is in 1NF and every non-key column depends fully on the entire primary key. In our 1NF table, StudentName and Email depend only on StudentID while ClubRoom and ClubMentor depend only on ClubName. These partial dependencies violate 2NF, and so the table is split into three separate tables.

StudentID (PK)	StudentName	Email
1	Asha	asha@email.com
2	Bikash	bikash@email.com
3	Nisha	nisha@email.com
4	Rohan	rohan@email.com
5	Suman	suman@email.com
6	Pooja	pooja@email.com
7	Aman	aman@email.com

Figure 11: Normalised Student Table (2NF)

ClubID (PK)	ClubName	ClubRoom	ClubMentor
C01	Music Club	R101	Mr. Raman
C02	Sports Club	R202	Ms. Sita
C03	Drama Club	R303	Mr. Kiran
C04	Coding Club	Lab1	Mr. Anil

Figure 12: Normalised Club Table (2NF)

MemberID (PK)	StudentID (FK)	ClubID (FK)	JoinDate
M01	1	C01	1/10/2024
M02	2	C02	1/12/2024
M03	1	C02	1/15/2024
M04	3	C01	1/20/2024
M05	4	C03	1/18/2024
M06	5	C01	1/22/2024
M07	2	C03	1/25/2024
M08	6	C02	1/27/2024
M09	3	C04	1/28/2024
M10	7	C04	1/30/2024

Figure 13: Normalised Membership Table (2NF)

A table is in Third Normal Form (3NF) if it is in 2NF and there are no transitive dependencies. As DataCamp explains, a transitive dependency occurs when a non-key attribute depends on another non-key attribute rather than directly on the primary key. In the Club table, ClubRoom and ClubMentor both depend directly on ClubID and not on each other. All three tables are therefore in 3NF, which according to Wikipedia is the level at which most relations are free of insertion, update, and deletion anomalies.

3.3 Entity Relationship Diagram

The ER diagram represents the relationships between the three entities in the normalised database. Student and Club are the two main entities. Membership is the associative entity, also called a junction or bridge table, that resolves the many-to-many relationship between them. StudentID in the Membership table is a foreign key referencing the Student table, and ClubID is a foreign key referencing the Club table.

Entity	Attributes	Cardinality	Relationship
Student	StudentID (PK), StudentName, Email	One-to-Many	StudentID references Membership
Club	ClubID (PK), ClubName, ClubRoom, ClubMentor	One-to-Many	ClubID references Membership
Membership	MembershipID (PK), StudentID (FK), ClubID (FK), JoinDate	Many-to-Many	Links Student and Club

Figure 14: Entity Relationship Diagram for Club Membership System

3.4 Basic SQL Operations

The following SQL statements create the three normalised tables:

```
-- Create Student Table
CREATE TABLE Student (
    StudentID   INT PRIMARY KEY,
```

```
    StudentName VARCHAR(100),
    Email       VARCHAR(100)
);

-- Create Club Table
CREATE TABLE Club (
    ClubID     VARCHAR(10) PRIMARY KEY,
    ClubName   VARCHAR(100),
    ClubRoom   VARCHAR(20),
    ClubMentor VARCHAR(100)
);

-- Create Membership Table
CREATE TABLE Membership (
    MemberID  VARCHAR(10) PRIMARY KEY,
    StudentID INT         REFERENCES Student(StudentID),
    ClubID    VARCHAR(10) REFERENCES Club(ClubID),
    JoinDate  DATE
);
```

Task 4: Insert and Select Queries

```
-- Insert a new student
INSERT INTO Student (StudentID, StudentName, Email)
VALUES (8, 'Priya', 'priya@email.com');

-- Insert a new club
INSERT INTO Club (ClubID, ClubName, ClubRoom, ClubMentor)
VALUES ('C05', 'Photography Club', 'Lab2', 'Ms. Devi');

SELECT * FROM Student;
SELECT * FROM Club;
```

Task 5: SQL JOIN Query

After normalisation, data is spread across multiple tables. To get a combined view, for example to see which student joined which club and when, it is necessary to JOIN the tables together. According to PopSQL, the JOIN operation links rows from different tables based on shared key values. The query below reassembles the normalised data into a meaningful view.

```
-- Display Student Name, Club Name, and Join Date
SELECT s.StudentName, c.ClubName, m.JoinDate
FROM   Membership m
JOIN   Student    s ON m.StudentID = s.StudentID
JOIN   Club       c ON m.ClubID    = c.ClubID;
```

Figure 15: SQL JOIN Operations and Reassembly of Normalised Data (source: original diagram)

3.5 Reflection: Benefits of Normalisation and the Need for JOINs

Before normalisation, the Music Club room and mentor information was stored in three separate rows. After normalisation, this information is stored once in the Club table. If the mentor changes, only one row needs to be updated, eliminating update anomalies and ensuring consistency across the entire database. A new club can be added to the Club table without needing any student membership data, and a student can leave a club without losing the club's details.

Normalisation spreads data across multiple tables to eliminate redundancy, which is efficient for storage and updates. However, retrieving a combined view of the data requires reassembling information from those tables. As DigitalOcean notes, a normalised database can actually improve query performance because each table contains less data for the database to scan. The SQL JOIN operation performs this reassembly by matching rows from the Student and Club tables to rows in the Membership table using the shared foreign key values. JOINs are therefore not a consequence of poor design: they are the deliberate and correct mechanism for querying a well-normalised database.

References

Task 1: Encoding Formats and Secure Data Exchange

[1] Josefsson, S. (2006). The Base16, Base32, and Base64 Data Encodings. RFC 4648. IETF. Available at: https://www.rfc-editor.org/rfc/rfc4648 [Accessed 1 March 2026].

[2] Forouzan, B.A. (2012). Data Communications and Networking. 5th edn. New York: McGraw-Hill.

[3] Klensin, J. (2008). Simple Mail Transfer Protocol. RFC 5321. IETF. Available at: https://www.rfc-editor.org/rfc/rfc5321 [Accessed 1 March 2026].

[4] OWASP (2021). OWASP Top 10: Injection. Available at: https://owasp.org/Top10/A03_2021-Injection/ [Accessed 2 March 2026].

[5] Rescorla, E. (2018). The Transport Layer Security (TLS) Protocol Version 1.3. RFC 8446. IETF. Available at: https://www.rfc-editor.org/rfc/rfc8446 [Accessed 1 March 2026].

[6] Built In (2024). Base64 Encoding: What Is It? How Does It Work? Available at: https://builtin.com/software-engineering-perspectives/base64-encoding [Accessed 5 March 2026].

[7] Wikipedia (2026). Base64. Available at: https://en.wikipedia.org/wiki/Base64 [Accessed 5 March 2026].

Task 2: P vs NP and Seating Arrangement Problem

[8] Sipser, M. (2012). Introduction to the Theory of Computation. 3rd edn. Boston: Cengage Learning.

[9] Garey, M.R. and Johnson, D.S. (1979). Computers and Intractability: A Guide to the Theory of NP-Completeness. San Francisco: W.H. Freeman.

[10] GeeksforGeeks (2024). P vs NP Problems. Available at: https://www.geeksforgeeks.org/dsa/p-vs-np-problems/ [Accessed 6 March 2026].

[11] Wikipedia (2026). P versus NP problem. Available at: https://en.wikipedia.org/wiki/P_versus_NP_problem [Accessed 6 March 2026].

Task 3: Database Normalisation, ER Modelling and SQL

[12] Codd, E.F. (1970). A Relational Model of Data for Large Shared Data Banks. Communications of the ACM, 13(6), pp.377-387.

[13] Date, C.J. (2003). An Introduction to Database Systems. 8th edn. Boston: Addison-Wesley.

[14] DigitalOcean (2024). Database Normalization: 1NF, 2NF, 3NF and BCNF Examples. Available at: https://www.digitalocean.com/community/tutorials/database-normalization [Accessed 7 March 2026].

[15] GeeksforGeeks (2025). Normal Forms in DBMS. Available at: https://www.geeksforgeeks.org/dbms/normal-forms-in-dbms/ [Accessed 7 March 2026].

[16] Wikipedia (2026). Database Normalization. Available at: https://en.wikipedia.org/wiki/Database_normalization [Accessed 7 March 2026].

[17] FreeCodeCamp (2022). Database Normalization: 1NF, 2NF, 3NF Table Examples. Available at: https://www.freecodecamp.org/news/database-normalization-1nf-2nf-3nf-table-examples/ [Accessed 7 March 2026].

Appendix: Project Repository

GitHub
Repository:
https://github.com/Ethex-wq/Foundation-of-Computer-Science-/blob/main/README.md

This repository contains the complete coursework submission for ST4015CMD Foundation of Computer Science at Softwarica College of IT and E-Commerce. It includes the full written report covering all three tasks: encoding formats and secure data exchange, P vs NP and the classroom seating problem, and database normalisation with SQL. Supporting materials such as SQL schema files, query scripts, normalisation diagrams, and the ER diagram are also included. The uploaded PDF report contains all figures, tables, and code referenced throughout the assignment.